

Advanced Techniques (4): Model Ensembling

Tutorials 1 and 2 improved the features; Tutorial 3 tuned a single model. In this notebook we keep the feature set fixed and combine *several* models.

Ensembling combines predictions from several models. The point is not to average many models for its own sake — it works only when different models make different kinds of errors. This notebook is about seeing *when* that happens and when it does not, and about a habit worth keeping: an ensemble is worth submitting only if cross-validation shows it beats the best single model it is built from.

Task: Predict the probability that an NFL prospect is **Drafted**. **1** means the player was drafted, and **0** means the player was not drafted.

Evaluation metric: ROC-AUC.

Connection to Previous Lectures

In this notebook, we apply ideas from the previous lectures to a competition-style ensemble workflow.

- From **Supervised Learning**, we compare several tree-based classification models.
- From **Model Evaluation**, we use ROC-AUC and out-of-fold predictions to compare models fairly.
- From **Feature Engineering**, we reuse the features created in Tutorials 1 and 2.
- From **Ensemble Learning**, we combine predictions from multiple models and check whether the combination improves validation performance.

Contents

1. Setup
2. Load the data
3. Understand the data
4. Preprocessing and feature set
5. Train several models
6. Inspect model diversity
7. Build ensembles
8. Create the submission file
9. Wrap-up

1. Setup

1.1 Import libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

import warnings
warnings.filterwarnings('ignore')

SEED = 42
np.random.seed(SEED)
```

```
# Install required libraries if they are not available in your environment.
!pip install -q xgboost lightgbm catboost
```

1.2 Connect with Google Drive

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
/content/drive/MyDrive/competition
```

All required files exist and the path is set correctly.

```
train: (2781, 16), test: (696, 15), sample sub: (696, 2)
```

	<u>Id</u>	<u>Year</u>	<u>Age</u>	<u>School</u>	<u>Height</u>	<u>Weight</u>	<u>Sprint_40yd</u>	<u>Vertical_Jump</u>	<u>Bench_Press_Reps</u>	<u>Broad_Jump</u>	<u>Agility_3cone</u>	<u>Shuttle</u>
0	0	2011	21.0	Lehigh	1.9050	140.160042	5.39	59.69	29.0	251.46	7.91	4.94
1	1	2011	24.0	Abilene Christian	1.8288	87.089735	4.31	101.60	16.0	332.74	NaN	NaN
2	2	2018	21.0	Colorado St.	1.8542	92.986436	4.51	91.44	10.0	309.88	6.95	4.37
3	3	2010	21.0	East	1.9304	148.778297	5.09	76.20	39.0	254.00	8.12	4.71

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2781 entries, 0 to 2780  
Data columns (total 16 columns):  
#   Column              Non-Null Count  Dtype
```

```

---
0  Id                2781 non-null  int64
1  Year              2781 non-null  int64
2  Age               2346 non-null  float64
3  School            2781 non-null  object
4  Height            2781 non-null  float64
5  Weight            2781 non-null  float64
6  Sprint_40yd       2636 non-null  float64
7  Vertical_Jump      2227 non-null  float64
8  Bench_Press_Reps  2060 non-null  float64
9  Broad_Jump         2200 non-null  float64
10 Agility_3cone     1811 non-null  float64
11 Shuttle           1869 non-null  float64
12 Player_Type       2781 non-null  object
13 Position_Type     2781 non-null  object
14 Position           2781 non-null  object
15 Drafted           2781 non-null  float64
dtypes: float64(10), int64(2), object(4)
memory usage: 347.8+ KB

```

3. Understand the data

Tutorials 1 and 2 handled the EDA and the features. Here the data step is brief — the focus is the models, not new features. The one rule to hold onto: the same features and the same CV split for every model, plus OOF predictions, so an ensemble can be judged fairly against the best single model.

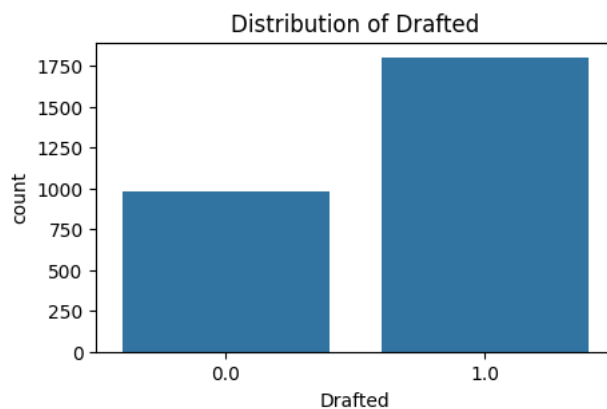
3.1 Target balance

```

plt.figure(figsize=(5, 3))
sns.countplot(data=train_raw, x='Drafted')
plt.title('Distribution of Drafted')
plt.show()

(train_raw['Drafted'].value_counts(normalize=True) * 100).round(2)

```



```

proportion

Drafted
1.0    64.83
0.0    35.17
dtype: float64

```

Findings:

- The target is imbalanced, so every model is compared on **OOF ROC-AUC**.

3.2 Feature groups reused from previous tutorials

```

combine_cols = [
    'Height', 'Weight', 'Sprint_40yd', 'Vertical_Jump',
    'Bench_Press_Reps', 'Broad_Jump', 'Agility_3cone', 'Shuttle'
]

```

```
train_raw[combine_cols].describe().T
```

	count	mean	std	min	25%	50%	75%	max	
Height	2781.0	1.873590	0.067178	1.651000	1.828800	1.879600	1.930400	2.082800	
Weight	2781.0	109.502026	20.307239	67.585263	92.986436	104.779837	124.284309	162.386068	
Sprint_40yd	2636.0	4.764818	0.299343	4.220000	4.530000	4.680000	4.950000	6.000000	
Vertical_Jump	2227.0	83.506349	10.710691	44.450000	76.200000	83.820000	91.440000	114.300000	
Bench_Press_Reps	2060.0	20.236408	6.410662	2.000000	16.000000	20.000000	24.000000	45.000000	
Broad_Jump	2200.0	291.985700	23.924666	198.120000	276.860000	294.640000	307.340000	373.380000	
Agility_3cone	1811.0	7.230447	0.410686	6.280000	6.930000	7.130000	7.490000	9.040000	
Shuttle	1869.0	4.399422	0.263851	3.820000	4.200000	4.350000	4.560000	5.560000	

Findings:

- The feature set is frozen (the features from the earlier tutorials), so any score difference here comes from the *model* or the *ensemble*, not the features.

4. Preprocessing and feature set

We reuse the frozen feature set from Tutorials 1 and 2. If each model saw different features we could not tell whether a gain came from the model, the features, or the ensemble. Fixing the features isolates the one question here: does combining models help?

4.1 Define preprocessing constants

```
TARGET = 'Drafted'
ID_COL = 'Id'
HIGH_CARDINALITY_COL = 'School'

NUMERIC_MISSING_COLS = [
    'Age',
    'Sprint_40yd',
    'Vertical_Jump',
    'Bench_Press_Reps',
    'Broad_Jump',
    'Agility_3cone',
    'Shuttle',
]

CATEGORICAL_COLS = ['Player_Type', 'Position_Type', 'Position']
```

4.2 Build the feature matrix

The helper below creates an all-numeric feature matrix.

The same preprocessing is applied to every model. This is important because we want to compare model behavior, not different feature sets.

```
def prepare_features(train_raw, test_raw):
    train = train_raw.copy()
    test = test_raw.copy()

    # Simple features from Tutorial 1
    train['BMI'] = train['Weight'] / (train['Height'] ** 2)
    test['BMI'] = test['Weight'] / (test['Height'] ** 2)

    school_counts = train[HIGH_CARDINALITY_COL].value_counts()
    train['School_CE'] = train[HIGH_CARDINALITY_COL].map(school_counts).fillna(0)
    test['School_CE'] = test[HIGH_CARDINALITY_COL].map(school_counts).fillna(0)

    for c in NUMERIC_MISSING_COLS:
        train[f'{c}_missing'] = train[c].isna().astype(int)
        test[f'{c}_missing'] = test[c].isna().astype(int)
```

```

# Mean imputation for numeric columns with missing values
for c in NUMERIC_MISSING_COLS:
    mean_value = train[c].mean()
    train[c] = train[c].fillna(mean_value)
    test[c] = test[c].fillna(mean_value)

# Domain features from Tutorial 2
train['SPEED_SCORE'] = (train['Weight'] * 200.0) / (train['Sprint_40yd'] ** 4)
test['SPEED_SCORE'] = (test['Weight'] * 200.0) / (test['Sprint_40yd'] ** 4)

vj_mean = train['Vertical_Jump'].mean()
vj_std = train['Vertical_Jump'].std()
bj_mean = train['Broad_Jump'].mean()
bj_std = train['Broad_Jump'].std()

train['EXPLOSIVENESS'] = 0.5 * (
    (train['Vertical_Jump'] - vj_mean) / vj_std
    + (train['Broad_Jump'] - bj_mean) / bj_std
)
test['EXPLOSIVENESS'] = 0.5 * (
    (test['Vertical_Jump'] - vj_mean) / vj_std
    + (test['Broad_Jump'] - bj_mean) / bj_std
)

train['AGILITY_RATIO'] = train['Agility_3cone'] / train['Shuttle'].where(train['Shuttle'] > 0, np.nan)
test['AGILITY_RATIO'] = test['Agility_3cone'] / test['Shuttle'].where(test['Shuttle'] > 0, np.nan)

ratio_median = train['AGILITY_RATIO'].median()
train['AGILITY_RATIO'] = train['AGILITY_RATIO'].fillna(ratio_median)
test['AGILITY_RATIO'] = test['AGILITY_RATIO'].fillna(ratio_median)

# Label encoding for low-cardinality categorical columns
for c in CATEGORICAL_COLS:
    categories = train[c].astype(str).unique()
    mapping = {cat: i for i, cat in enumerate(categories)}

    train[c] = train[c].astype(str).map(mapping).astype(int)
    test[c] = test[c].astype(str).map(mapping).fillna(-1).astype(int)

drop_cols = [ID_COL, HIGH_CARDINALITY_COL, TARGET]
feature_cols = [c for c in train.columns if c not in drop_cols]

X = train[feature_cols].copy()
y = train[TARGET].astype(int)
X_test = test[feature_cols].copy()

return X, y, X_test, feature_cols

```

```
X, y, X_test, feature_cols = prepare_features(train_raw, test_raw)
```

```
print(f'Number of features: {len(feature_cols)}')
print(feature_cols)
```

```
Number of features: 25
['Year', 'Age', 'Height', 'Weight', 'Sprint_40yd', 'Vertical_Jump', 'Bench_Press_Reps', 'Broad_Jump', 'Agility_3cone', 'Shuttle']
```

5. Train several models

An ensemble is useful only if the base models are reasonably strong and make somewhat different predictions.

In this section, we train several boosting models with the same feature matrix and the same cross-validation split.

5.1 Define cross-validation helper

```

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

def cv_fit_predict(make_model, X, y, X_test=None):

```

```

oof = np.zeros(len(X))
test_pred = np.zeros(len(X_test)) if X_test is not None else None
fold_aucs = []

for fold, (tr_idx, va_idx) in enumerate(skf.split(X, y), start=1):
    X_tr, X_va = X.iloc[tr_idx].copy(), X.iloc[va_idx].copy()
    y_tr, y_va = y.iloc[tr_idx], y.iloc[va_idx]

    model = make_model()
    model.fit(X_tr, y_tr)

    va_pred = model.predict_proba(X_va)[:, 1]
    oof[va_idx] = va_pred

    fold_auc = roc_auc_score(y_va, va_pred)
    fold_aucs.append(fold_auc)

    if X_test is not None:
        test_pred += model.predict_proba(X_test)[:, 1] / skf.n_splits

    print(f'Fold {fold}: AUC = {fold_auc:.4f}')

oof_auc = roc_auc_score(y, oof)
print(f'OOF AUC: {oof_auc:.4f}')

return oof, test_pred, oof_auc, fold_aucs

```

5.2 Define models

We use four boosting models.

- **AdaBoost:** a classic boosting method that combines many weak learners.
- **XGBoost:** a widely used gradient boosting implementation.
- **LightGBM:** a fast gradient boosting implementation.
- **CatBoost:** a boosting model that often works well with tabular data.

The exact hyperparameters are not the main point of this notebook. The main point is to obtain several sets of OOF predictions and compare how they can be combined.

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

def make_adaboost():
    return AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1, random_state=SEED),
        n_estimators=500,
        learning_rate=0.05,
        random_state=SEED,
    )

def make_xgboost():
    return XGBClassifier(
        objective='binary:logistic',
        eval_metric='auc',
        n_estimators=600,
        learning_rate=0.05,
        max_depth=5,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_weight=10,
        reg_lambda=1.0,
        tree_method='hist',
        n_jobs=-1,
        random_state=SEED,
    )

def make_lightgbm():

```

```

    return LGBMClassifier(
        objective='binary',
        n_estimators=1000,
        learning_rate=0.03,
        num_leaves=63,
        subsample=0.8,
        colsample_bytree=0.8,
        min_child_samples=40,
        n_jobs=-1,
        random_state=SEED,
        verbose=-1,
    )

def make_catboost():
    return CatBoostClassifier(
        loss_function='Logloss',
        eval_metric='AUC',
        iterations=1000,
        learning_rate=0.03,
        depth=6,
        l2_leaf_reg=3.0,
        random_seed=SEED,
        verbose=False,
        allow_writing_files=False,
    )

```

5.3 Evaluate each model

```

model_factories = {
    'AdaBoost': make_adaboost,
    'XGBoost': make_xgboost,
    'LightGBM': make_lightgbm,
    'CatBoost': make_catboost,
}

oof_predictions = {}
test_predictions = {}
model_rows = []

for model_name, make_model in model_factories.items():
    print('\n' + '=' * 80)
    print(model_name)
    print('=' * 80)

    oof, test_pred, auc, fold_aucs = cv_fit_predict(
        make_model,
        X,
        y,
        X_test,
    )

    oof_predictions[model_name] = oof
    test_predictions[model_name] = test_pred

    model_rows.append({
        'model': model_name,
        'OOF AUC': auc,
        'fold_auc_mean': np.mean(fold_aucs),
        'fold_auc_std': np.std(fold_aucs),
    })

model_comparison = pd.DataFrame(model_rows).sort_values('OOF AUC', ascending=False)
model_comparison.style.format({
    'OOF AUC': '{:.4f}',
    'fold_auc_mean': '{:.4f}',
    'fold_auc_std': '{:.4f}',
})

```

```

=====
AdaBoost
=====
Fold 1: AUC = 0.7885
Fold 2: AUC = 0.8274
Fold 3: AUC = 0.8501
Fold 4: AUC = 0.7766
Fold 5: AUC = 0.8356
OOF AUC: 0.8125

=====
XGBoost
=====
Fold 1: AUC = 0.7928
Fold 2: AUC = 0.8433
Fold 3: AUC = 0.8246
Fold 4: AUC = 0.8067
Fold 5: AUC = 0.8307
OOF AUC: 0.8195

=====
LightGBM
=====
Fold 1: AUC = 0.7920
Fold 2: AUC = 0.8250
Fold 3: AUC = 0.8021
Fold 4: AUC = 0.7851
Fold 5: AUC = 0.8307
OOF AUC: 0.8066

=====
CatBoost
=====
Fold 1: AUC = 0.8034
Fold 2: AUC = 0.8556
Fold 3: AUC = 0.8232
Fold 4: AUC = 0.8067
Fold 5: AUC = 0.8404
OOF AUC: 0.8248

```

	model	OOF AUC	fold_auc_mean	fold_auc_std
3	CatBoost	0.8248	0.8259	0.0199
1	XGBoost	0.8195	0.8196	0.0179
0	AdaBoost	0.8125	0.8156	0.0282
2	LightGBM	0.8066	0.8070	0.0180

The best single model is the benchmark for the ensemble.

If an ensemble does not beat the best single model, it does not necessarily mean the ensemble is useless. It may still be more stable, but we should not assume that averaging always improves the score.

Questions to think about:

- Which single model is strongest, and is its lead larger than `fold_auc_std`?
- The hyperparameters are hand-set. Would tuning them (Tutorial 3) change the ranking of the models?

6. Inspect model diversity

An ensemble only helps if the base models are reasonably strong **and** make somewhat different errors. Before averaging anything, we inspect how similar the models are by correlating their out-of-fold (OOF) predictions.

- A **high** correlation means two models rank players in almost the same way, so averaging them adds little.
- A **lower** correlation between two *strong* models suggests they make different kinds of errors, which is exactly when ensembling tends to help.

Low correlation is not automatically good, though: a weak, noisy model can also look uncorrelated. A useful ensemble usually needs both **strong** individual models and **some diversity** among them. The correlation matrix below helps us inspect the second point.

```

oof_df = pd.DataFrame(oof_predictions)

plt.figure(figsize=(6, 5))

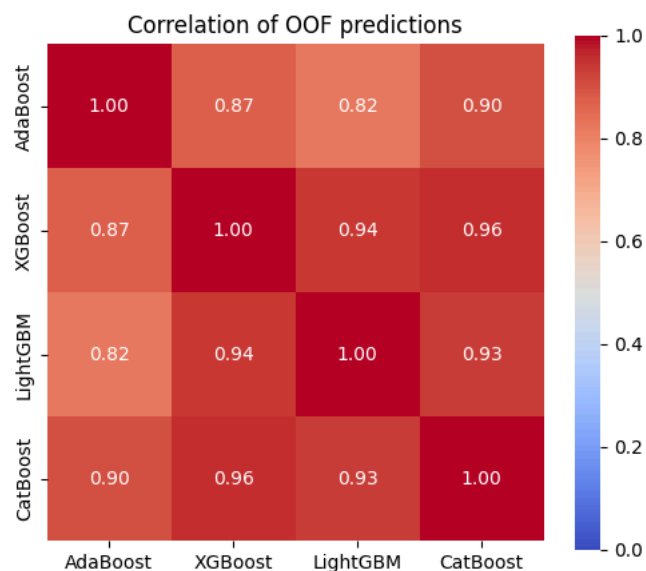
```

```

sns.heatmap(
    oof_df.corr(),
    annot=True,
    fmt='.2f',
    cmap='coolwarm',
    vmin=0,
    vmax=1,
    square=True,
)
plt.title('Correlation of OOF predictions')
plt.show()

oof_df.corr()

```



	AdaBoost	XGBoost	LightGBM	CatBoost
AdaBoost	1.000000	0.870368	0.823452	0.899716
XGBoost	0.870368	1.000000	0.942888	0.957934
LightGBM	0.823452	0.942888	1.000000	0.931424
CatBoost	0.899716	0.957934	0.931424	1.000000

Questions to think about:

- If two models correlate at ~0.99, what do you expect averaging them to achieve?
- A weak but uncorrelated model: is it a useful source of diversity, or just noise?

7. Build ensembles

Now we combine the model predictions. We try two simple, robust schemes and compare each against the **best single model** using OOF ROC-AUC:

- **Equal average** — the mean of the predicted probabilities.
- **Rank average** — average the per-model rankings, which is robust when models use different probability scales.

We deliberately avoid tuning ensemble weights on the same OOF predictions we score on, because that can overfit the validation data (we show this only as a diagnostic in 7.3).

7.1 Equal average and rank average

```

from scipy.stats import rankdata

model_names = list(oof_predictions.keys())

S_oof = np.column_stack([oof_predictions[m] for m in model_names])
S_test = np.column_stack([test_predictions[m] for m in model_names])

```

```
# Equal average of predicted probabilities
equal_oof = S_oof.mean(axis=1)
equal_test = S_test.mean(axis=1)
equal_auc = roc_auc_score(y, equal_oof)

# Rank average of predictions
rank_oof = np.column_stack([rankdata(S_oof[:, i]) for i in range(S_oof.shape[1])]).mean(axis=1)
rank_test_raw = np.column_stack([rankdata(S_test[:, i]) for i in range(S_test.shape[1])]).mean(axis=1)

# Rescale rank-averaged test predictions to [0, 1] for submission.
rank_test = (rank_test_raw - rank_test_raw.min()) / (rank_test_raw.max() - rank_test_raw.min())
rank_auc = roc_auc_score(y, rank_oof)

ensemble_rows = [
    {'method': 'Equal average ensemble', 'OOF AUC': equal_auc},
    {'method': 'Rank average ensemble', 'OOF AUC': rank_auc},
]

ensemble_comparison = pd.DataFrame(ensemble_rows).sort_values('OOF AUC', ascending=False)
ensemble_comparison.style.format({'OOF AUC': '{:.4f}'})
```

	method	OOF AUC
1	Rank average ensemble	0.8267
0	Equal average ensemble	0.8232

7.2 Compare single models and ensembles

```
score_rows = []
submission_candidates = {}

for model_name in model_names:
    auc = roc_auc_score(y, oof_predictions[model_name])
    score_rows.append({'method': model_name, 'OOF AUC': auc})
    submission_candidates[model_name] = test_predictions[model_name]

score_rows.append({'method': 'Equal average ensemble', 'OOF AUC': equal_auc})
submission_candidates['Equal average ensemble'] = equal_test

score_rows.append({'method': 'Rank average ensemble', 'OOF AUC': rank_auc})
submission_candidates['Rank average ensemble'] = rank_test

score_table = pd.DataFrame(score_rows).sort_values('OOF AUC', ascending=False)
score_table.style.format({'OOF AUC': '{:.4f}'})
```

	method	OOF AUC
5	Rank average ensemble	0.8267
3	CatBoost	0.8248
4	Equal average ensemble	0.8232
1	XGBoost	0.8195
0	AdaBoost	0.8125
2	LightGBM	0.8066

7.3 Optional diagnostic: learned weights

The following cell is optional. It shows why learned ensemble weights should be treated carefully.

The weights are optimized on the same OOF predictions used for evaluation, so the result may be overly optimistic. We calculate it only as a diagnostic and do not use it for the submission.

```
from scipy.optimize import minimize

def negative_auc(weights):
    weights = np.clip(weights, 0, None)
    if weights.sum() == 0:
```

```

        return 0.0
    blended = (S_oof * weights).sum(axis=1) / weights.sum()
    return -roc_auc_score(y, blended)

result = minimize(
    negative_auc,
    x0=np.ones(len(model_names)) / len(model_names),
    method='Nelder-Mead',
)

learned_weights = np.clip(result.x, 0, None)
learned_weights = learned_weights / learned_weights.sum()
learned_auc = -result.fun

weights_df = pd.DataFrame({
    'model': model_names,
    'learned_weight': learned_weights,
}).sort_values('learned_weight', ascending=False)

display(weights_df)

print(f'Learned-weight OOF AUC: {learned_auc:.4f}')
print('This is a diagnostic only and is not used for submission.')

```

	model	learned_weight
0	AdaBoost	0.501907
3	CatBoost	0.417921
1	XGBoost	0.080172
2	LightGBM	0.000000

Learned-weight OOF AUC: 0.8288
This is a diagnostic only and is not used for submission.

Next steps:

[Generate code with weights_df](#)[New interactive sheet](#)**Questions to think about:**

- Did any ensemble beat the best single model on OOF AUC? If not, what does that tell you about the models?
- The learned weights scored highest — but on the *same* OOF data used to score them. Why is that optimistic?
- Would **stacking** (training a small meta-model on the OOF predictions, inside CV) be a safer way to learn weights than hand-tuning them?

8. Create the submission file

We use the method with the best OOF AUC from the comparison table.

If an ensemble is selected, that is because it improved validation performance. If a single model is selected, that is also a valid outcome: ensembling does not always beat the best individual model.

```

selected_method = score_table.iloc[0]['method']
print('Selected submission method:', selected_method)

test_pred = submission_candidates[selected_method]
print('test_pred shape:', test_pred.shape)

```

Selected submission method: Rank average ensemble
test_pred shape: (696,)

Saving the prediction as a CSV file [DO NOT CHANGE]

WARNING: Do not change the following cell unless you understand what it is doing.

The final CSV must exactly match the expected format. By using `sample_submission.csv`, we preserve the required row order and column structure. Here, we replace only the `Drafted` column with our predictions.

```

submission = pd.read_csv(sample_sub_file)
submission['Drafted'] = test_pred

```

```

out_dir = WORK_DIR / 'output'
out_dir.mkdir(exist_ok=True)

submission_path = out_dir / 'submission.csv'
submission.to_csv(submission_path, index=False)

print('Saved to:', submission_path)
submission.head()

```

Saved to: /content/drive/MyDrive/competition/output/submission.csv

	Id	Drafted	
0	2781	0.265306	
1	2782	0.694971	
2	2783	0.911443	
3	2784	0.884840	
4	2785	0.688047	

Next steps:

[Generate code with submission](#)

[New interactive sheet](#)

9. Wrap-up

In this notebook we trained several models on one frozen feature set, looked at how differently they rank players, and combined them — always measuring against the best *single* model.

An ensemble only earns its place when it beats the best model it is made of.

Questions to think about:

- You have now worked through features (1–2), tuning (3), and ensembling (4). Which stage moved your CV score the most on this dataset?
- Could you feed a **tuned** CatBoost from Tutorial 3 into this ensemble instead of the default one?
- Is your final submission chosen by **cross-validation**, rather than by chasing the public leaderboard?

*If a cell raises an error, re-run everything from **Section 2** so the raw data is rebuilt.*